

A Appendix

A.1 Example Observations with actions from the Game *Realsokoban*

For visual clarity, observations are illustrated using a compact game from the GVGAI environment, as the representative games' grids exceed the page layout constraints.

```

1
2 wall wall wall wall wall wall wall wall
3 wall wall
4 wall wall hole box hole wall
5 wall wall box avatar box wall
6 wall hole box hole wall
7 wall wall
8 wall wall wall wall wall wall wall wall
9
10 Action: Down
11
12 wall wall wall wall wall wall wall wall
13 wall wall
14 wall wall hole box hole wall
15 wall wall box box wall
16 wall hole avatar hole wall
17 wall box wall
18 wall wall wall wall wall wall wall wall
19
20 Action: Up
21
22 wall wall wall wall wall wall wall wall
23 wall wall
24 wall wall hole box hole wall
25 wall wall box avatar box wall
26 wall hole hole wall
27 wall box wall
28 wall wall wall wall wall wall wall wall

```

A.2 Preliminary Model Evaluation

We evaluated varying models from cost-effective parameter categories (sub-4B and sub-8B) to ensure computational efficiency. As illustrated in Figure 7, the **Qwen3-8B** model emerged as a significant outlier compared to other tested architectures.

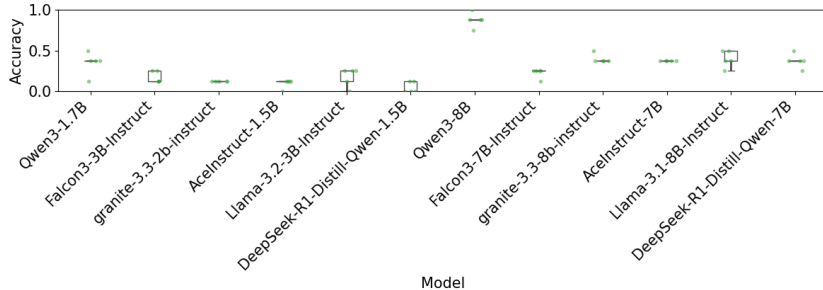


Fig. 7. Preliminary accuracy assessment for a 10-way game classification task (using random games from [9]). The Qwen3-8B model demonstrates significantly superior performance compared to other sub-4B and sub-8B models, justifying its selection for the VGDL translation task.

A.3 Example natural language description generated by Qwen3-8B from VGDL of the game *Brainman*

In *Brainman*, the player controls an avatar navigating a maze. Collect keys (turning into missiles) to unlock doors and defeat enemies (gems) for points. Boulders bounce on contact, and walls/doors block movement. The avatar loses if killed by gems (1-5 points) or trapped. Winning requires reaching the exit (10 points). Key missiles destroy walls, while undo interactions reset objects. The game ends when the avatar dies or the exit is reached.

A.4 Clustering Hyperparameters and Preprocessing

To determine the optimal number of semantic clusters, we performed a sweep over $k \in [5, 14]$, evaluating both *Inertia* and *Silhouette Score*. As shown in Figure 8, the Silhouette analysis identifies a distinct global maximum at $k = 9$.

Furthermore, we investigated the stability of these clusters under different preprocessing strategies. Although the official S-BERT example code implements K-Means directly on raw embeddings [1], we expanded our analysis to include Standard scaling [22], L2 normalization, and dimensionality reduction (PCA [21], UMAP) for completeness.

Figures 9–11 illustrate the Elbow and Silhouette metrics across these exhaustive configurations. We observed that while dimensionality reduction and scaling can aid in visualization, the raw, 384-dimensional embeddings provided robust cluster separation without the information loss associated with projection, validating our decision to follow the official documentation’s approach for the main analysis.

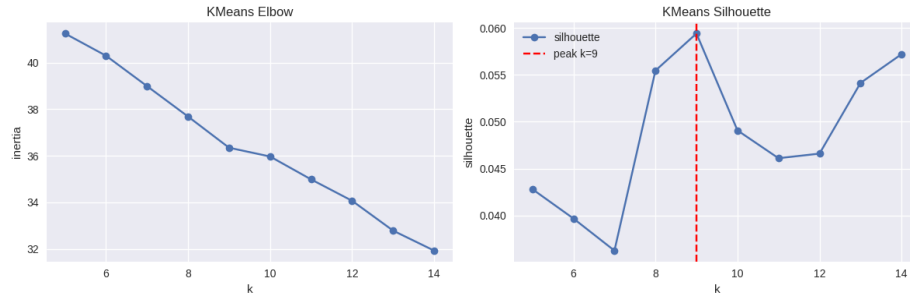


Fig. 8. Evaluation of K-Means clustering metrics. The Elbow method (left) displays inertia reduction, while the Silhouette analysis (right) confirms $k = 9$ as the optimal parameter.

KMeans Analysis - Part 1 of 3

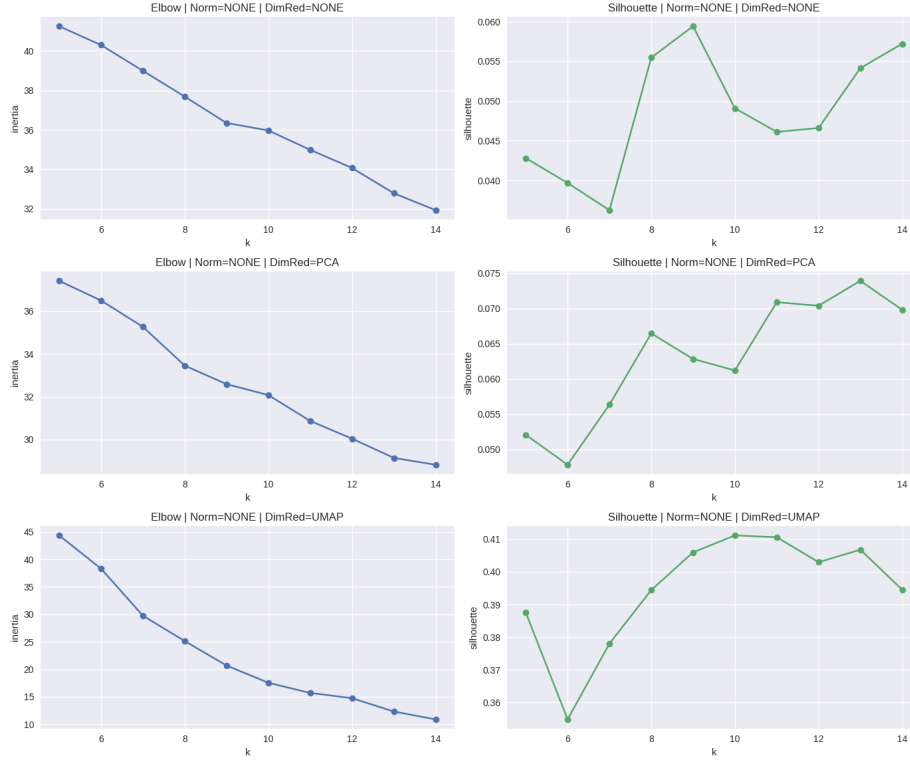


Fig. 9. Clustering performance analysis (Part 1): Comparison of Inertia and Silhouette scores across different preprocessing combinations, including Standard scaling [22] and L2 Normalization.

KMeans Analysis - Part 2 of 3

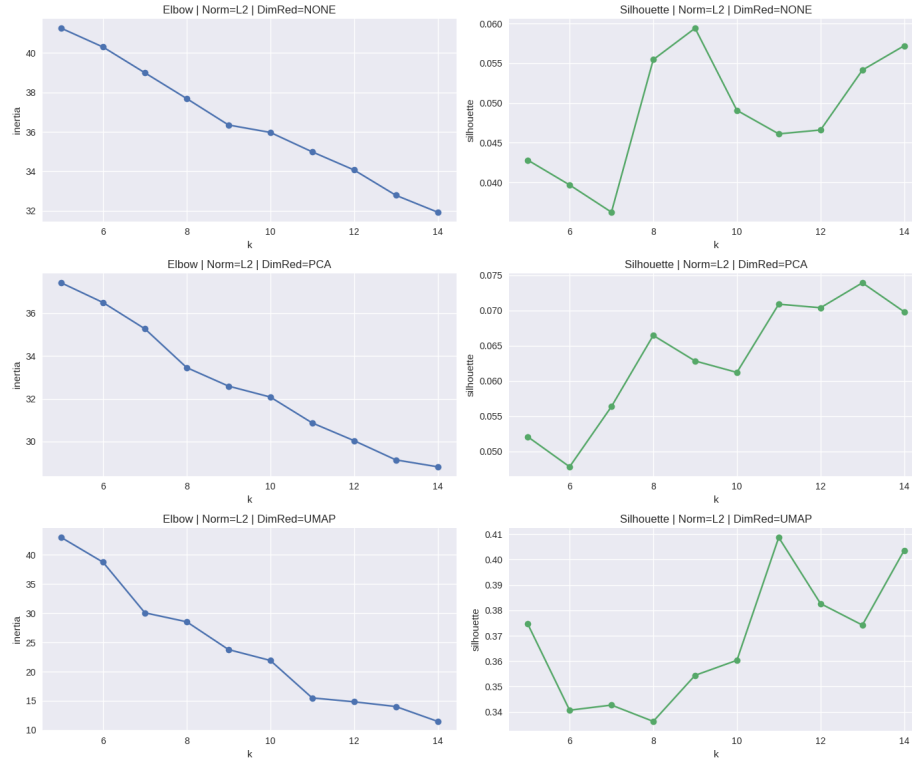


Fig. 10. Clustering performance analysis (Part 2): Extended evaluation of dimensionality reduction techniques (PCA) combined with scaling strategies.

KMeans Analysis - Part 3 of 3

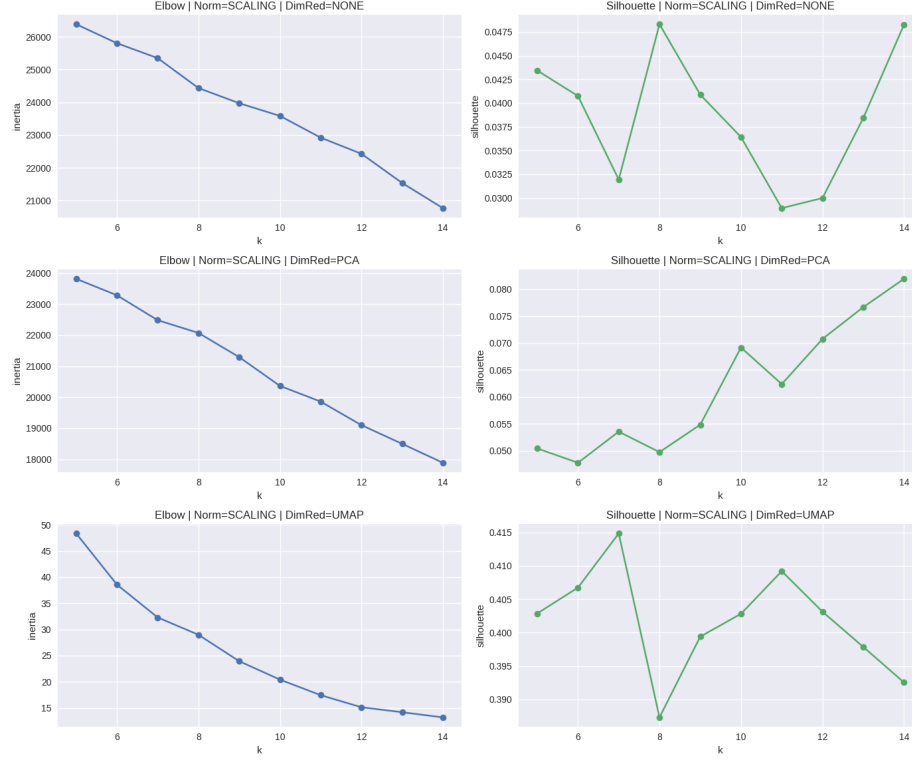


Fig. 11. Clustering performance analysis (Part 3): Extended evaluation of manifold learning techniques (UMAP) combined with scaling strategies.

A.5 Clusters made using K-means clustering

Table 3: Extended KMeans clustering results with all cluster members.

Cluster ID	Representative game	Members
00	racebet	racebet, camelRace, racebet2
01	digdug	superman, islands, rivers, chase, plants, frogs, eggomania, digdug, plaqueattack, painter, sheriff, pacman, gymkhana, surround, bait, overload, whackamole, seaquest

Continued on next page

Table 3 – *Continued from previous page*

Cluster ID	Representative game	Members
02	brainman	brainman, roguelike, escape, labyrinth-dual, intersection, realsokoban, avoidgeorge, lemmings, run, freeway, enemycitadel, blacksmoke, labyrinth, sokoban, zelda, thecitadel, clusters, waitforbreakfast, cops, dungeon
03	defender	defender, waves, aliens, missilecommand, solarfox
04	lasers2	boloadventures, assemblyline, lasers2, factorymanager, lasers
05	boulderchase	shipwreck, boulderdash, jaws, boulderchase, chainreaction, butterflies
06	portals	portals, colourescape, crossfire, witnessprotection, zenpuzzle, realportals, chopper, catapults, tercio, bomber, infection, modality, angelsdemons
07	iceandfire	chipschallenge, survivezombies, firestorms, thesnowman, iceandfire, fireman, firecaster
08	cakybaky	cookmepasta, hungrybirds, cakybaky

A.6 Game descriptions provided by expert

Racebet: The objective of this game is guessing which camel is going to win the race. Betting is performed by placing the player in one of the corresponding betting spots. No points are given in this game. You either bet for the one that wins (and the player wins) or you don't (and the player loses).

Digdug: The player must collect all gems and gold coins in the cave, digging its way through it, to win the game. There are also enemies in the level that kill the player on collision with them. Also, the player can shoot boulders by pressing USE two consecutive time steps, which kill enemies.

Brainman: The player must push keys into doors to open them, to reach the exit to win the game. Many gems around are available for collection, which give score to the player. Keys are pushed by colliding with them, and they are propelled in a straight line until finding a sprite to collide against, when they stop. If they hit a door, both the door and the key are destroyed.

Defender: The objective of this game is to avoid that the enemies destroy your city. The enemies are spawned at the right of the screen and cross it while bombing the city. The player must eliminate these enemies as soon as possible so the city is not destroyed, otherwise the player loses. The player's ammunition is supplied from the top of the screen and must be collected periodically to avoid running out of ammo.

lasers2: The objective of this game is to reach the exit, which is typically blocked by a mud sprite. The mud can be destroyed by directing a laser to it. There are several laser cannons whose orientation can be changed by the player shooting at them. Apart from this, there are several crystal blocks, which can also be rotated, that deflect the lasers in a certain direction. Lasers kill the player when in contact.

Boulderchase: In this game, the player must dig inside a cave to collect at least 9 gems and reach the exit. As the player digs, boulders that are left without a ground fall and can kill the player upon collision. Enemies also move around the cave, being able to dig as well. When digging, they drop gems that can also be collected by the player. Enemies are also killed by boulders and kill the player upon contact.

Portals: The player controls an avatar that needs to find the exit of a maze. The maze is full of entry and exit portals, and every time the player goes through an entry portal they will be teleported to an exit portal at random. The maze is also full of enemies and missiles that kill the player on contact.

Iceandfire: The objective of this game is to find the exit of the maze. The maze has multiple traps, two different types of surfaces (ice and fire), and two types of boots (for ice and fire) that can be collected. The special surfaces kill the player on contact, unless the appropriate boots are picked up beforehand. There are several coins scattered around the level that give the player some score.

A.7 Example of a ground-truth VGD_L for the game *Brainman*

```

1 BasicGame
2   SpriteSet
3     floor > Immovable hidden=True img=newset/floor2
4     avatar > OrientedAvatar img=oryx/prince1 rotateInPlace=false
5     gem > Immovable shrinkfactor=0.7
6       green > img=oryx/diamond3
7       red > img=oryx/diamond2
8       blue > img=oryx/diamond1
9     key > Passive img=oryx/key3 shrinkfactor=0.7
10    keym > Missile img=oryx/key3 shrinkfactor=0.6
11    exit > Immovable img=newset/exit2
12    door > Immovable img=oryx/doorclosed1
13    boulder > Passive img=newset/block3
14    wall > Immovable img=oryx/wall3 autotiling=True
15
16   LevelMapping
17     A > avatar floor
18     k > key floor
19     d > door floor
20     e > exit floor
21     g > green floor
22     r > red floor
23     b > blue floor
24     0 > boulder floor
25     . > floor
26
27   InteractionSet
28     keym key wall gem boulder > transformTo stype=key
29     avatar wall door > stepBack
30     boulder avatar > bounceForward
31     key avatar > transformTo stype=keym
32     keym avatar > attractGaze
33     avatar key keym > stepBack
34     door keym > killBoth scoreChange=4
35     green avatar > killSprite scoreChange=1
36     blue avatar > killSprite scoreChange=2
37     red avatar > killSprite scoreChange=5
38     boulder wall key gem boulder > undoAll
39     key wall gem key > undoAll
40     exit avatar > killSprite scoreChange=10
41
42   TerminationSet
43     SpriteCounter stype=avatar limit=0 win=False
44     SpriteCounter stype=exit limit=0 win=True

```

A.8 SCM blueprint


```

1 nodes={
2   # Design Layer
3   "EntityTypes": "Static sprite types, roles (avatar, wall), & attrs (
4     speed, hp). Not time-varying.",
5   "ActionSpace": "Agent interventions/actions available at each step (
6     move, shoot).",
7   "GlobalMechanics": "Global state transitions independent of
8     collisions (gravity, timers).",
9   "InteractionMechanics": "Collision rules mapping (Entities, State_t,
10     Action_t) -> State_{t+1} events (spawn, destroy).",
11   "RewardMechanics": "Maps State_t/Interactions to Reward_t or Score_{t
12     +1}.",
13   "TerminationMechanics": "Win/loss conditions based on StateVariables
14     (e.g., timeout, goals_met).",
15   # Dynamics Layer
16   "StateVariables": "Dynamic vars evolving over t (pos, hp). Eq:  $X_{t+1}=f(PA_{X_t}, Action_t, U_X)$ .",
17   "InitialState": "StateVariables at t=0 derived from EntityTypes +
18     LevelEncoding.",
19   # Observation Layer
20   "LevelEncoding": "ASCII grid mapping chars to entity instances/
21     positions; induces InitialState.",
22 },
23 edges=[
24   ("EntityTypes", "InitialState"), ("LevelEncoding", "InitialState"), (
25     "EntityTypes", "StateVariables"),
26   ("ActionSpace", "InteractionMechanics"), ("ActionSpace", "
27     StateVariables"),
28   ("GlobalMechanics", "StateVariables"), ("InteractionMechanics", "
29     StateVariables"),
30   ("StateVariables", "InteractionMechanics"), ("StateVariables", "
31     RewardMechanics"),
32   ("InteractionMechanics", "RewardMechanics"), ("RewardMechanics", "
33     StateVariables"),
34   ("StateVariables", "TerminationMechanics"), ("InteractionMechanics",
35     "TerminationMechanics"),
36 ],
37 scm_notes=[
38   "Define DYNAMIC SCM with discrete time steps t=0,1...",
39   "Separate STATIC design vars from DYNAMIC state vars.",
40   "For dynamic X, define  $X_{t+1} = f(Parents_t, Action_t, Noise)$ .",
41   "Define Reward_t & Termination explicitly as equations.",
42   "Output machine-readable format (JSON) with: static_nodes,
43     dynamic_variables, edges, equations.",
44   "Link LevelEncoding chars to InitialState.",
45 ],
46 vgdL_notes=[
47   "EntityTypes+StateVariables -> VGDL SpriteSet (roles, attrs).",
48   "LevelEncoding+InitialState -> VGDL LevelMapping.",
49   "InteractionMechanics+GlobalMechanics -> VGDL InteractionSet.",
50   "RewardMechanics+TerminationMechanics -> VGDL TerminationSet.",
51   "ActionSpace -> VGDL Avatar controls.",
52 ],

```

Listing 1.1. SCM Blueprint Definition

A.9 Game-Specific Analysis for Task II

Table 4 details the performance breakdown by game, highlighting the specific code and semantic similarity scores for both generation streams.

Notably, the *portals* environment presented a distinct challenge, resulting in a **90% tie rate** for QwQ-32B. This was largely due to the extensive length of the game’s observations, which strained the model’s generation capabilities. As a result, the model frequently struggled to produce executable VGDL in either condition. The high tie rate, therefore, reflects a shared difficulty in handling this heavy token load, rather than an equivalence in successful reasoning.

Table 4. Game-Specific Metrics. Comparison of Code and Semantic similarities across key games for Non-SCM (NS) and SCM-Guided (SCM) streams.

Model	Game	NS-Code ↑	NS-Sem ↑	SCM-Code ↑	SCM-Sem ↑	NS Win% ↑	SCM Win% ↑	Tie% ↑
Qwen3-8B	boulderchase	0.788	0.777	0.816	0.783	31.2%	68.8%	0.0%
Qwen3-8B	brainman	0.664	0.775	0.668	0.788	15.0%	85.0%	0.0%
Qwen3-8B	cakybaky	0.747	0.801	0.739	0.791	25.0%	75.0%	0.0%
Qwen3-8B	defender	0.631	0.656	0.646	0.679	30.0%	70.0%	0.0%
Qwen3-8B	digdug	0.720	0.806	0.764	0.806	35.0%	60.0%	5.0%
Qwen3-8B	iceandfire	0.766	0.743	0.797	0.776	37.5%	62.5%	0.0%
Qwen3-8B	lasers2	0.647	0.757	0.741	0.809	20.0%	75.0%	5.0%
Qwen3-8B	portals	0.767	0.725	0.786	0.721	25.0%	75.0%	0.0%
Qwen3-8B	racebet	0.703	0.807	0.722	0.820	35.0%	60.0%	5.0%
QwQ-32B	boulderchase	0.815	0.795	0.802	0.826	40.0%	55.0%	5.0%
QwQ-32B	brainman	0.675	0.796	0.727	0.750	35.0%	65.0%	0.0%
QwQ-32B	cakybaky	0.720	0.775	0.756	0.778	10.0%	90.0%	0.0%
QwQ-32B	defender	0.609	0.705	0.673	0.739	40.0%	60.0%	0.0%
QwQ-32B	digdug	0.740	0.789	0.755	0.789	20.0%	75.0%	5.0%
QwQ-32B	iceandfire	0.757	0.797	0.761	0.843	45.0%	50.0%	5.0%
QwQ-32B	lasers2	0.684	0.818	0.775	0.752	15.0%	75.0%	10.0%
QwQ-32B	portals	0.367	0.591	0.245	0.649	0.0%	10.0%	90.0%
QwQ-32B	racebet	0.706	0.850	0.738	0.858	31.6%	57.9%	10.5%

A.10 Statistical Significance Testing

To rigorously evaluate the qualitative preference for the Structural Causal Model (SCM) guided pipeline (Stream B) over Direct Synthesis (Stream A), we performed a statistical analysis on the aggregate judge preferences from Task II. Given the large sample size of decisive trials ($n > 100$), we utilized the Normal Approximation to the Binomial Distribution.

Methodology We evaluate against the null hypothesis (H_0) that there is no underlying preference between the two methods, meaning the probability of an SCM win is $p_0 = 0.5$. For this analysis, we exclude ties, focusing strictly on decisive match-ups where a judge indicated a clear preference.

The expected mean (μ) and standard deviation (σ) under the null hypothesis are calculated as:

$$\mu = n \cdot p_0 \quad (1)$$

$$\sigma = \sqrt{n \cdot p_0 \cdot (1 - p_0)} \quad (2)$$

The Z-score is then derived using the observed number of SCM wins (x):

$$Z = \frac{x - \mu}{\sigma} \quad (3)$$

Qwen3-8B Analysis Across all four evaluation judges, the Qwen3-8B generation tasks yielded $n = 165$ decisive trials, with the SCM approach winning $x = 118$ times.

Applying the normal approximation:

- **Expected Mean:** $\mu = 165 \cdot 0.5 = 82.5$
- **Standard Deviation:** $\sigma = \sqrt{165 \cdot 0.5 \cdot 0.5} \approx 6.4226$
- **Z-score:** $Z = \frac{118-82.5}{6.4226} \approx 5.527$

A Z-score of 5.527 corresponds to a p -value of approximately 3×10^{-8} . Because this is significantly lower than standard significance thresholds, we reject the null hypothesis ($p < 0.001$).

QwQ-32B Analysis For the QwQ-32B model, the generation tasks yielded $n = 153$ decisive trials (due to a higher incidence of ties), with the SCM approach winning $x = 106$ times.

Applying the normal approximation:

- **Expected Mean:** $\mu = 153 \cdot 0.5 = 76.5$
- **Standard Deviation:** $\sigma = \sqrt{153 \cdot 0.5 \cdot 0.5} \approx 6.1846$
- **Z-score:** $Z = \frac{106-76.5}{6.1846} \approx 4.770$

A Z-score of 4.770 corresponds to a p -value of approximately 1.8×10^{-6} . We again reject the null hypothesis, confirming the preference for SCM is statistically significant ($p < 0.001$).